

Loops, Arrays, and Number Sequences

Topics: while-loop, do-while loop, for-loop, arrays

Lab 2: https://maryash.github.io/135/labs/lab_02.html

Ryan Vaz

Introducing the While-loop:

For-loops allow us to loop things for a certain number of times. However, there are situations when we need to repeat a task indefinitely as long as a certain condition is met every repetition.

We don't necessarily know how many times the task needs to be repeated but we know when it should repeat. In those cases, a while-loop is used:

```
4  while(condition){  
5      |    ... run code  
6  }  
7  
8  // the while loop will continue to run  
9  // until the condition above becomes false
```

Infinite loop and keyword `break`

A while-loop should always allow an exit. If the condition is met every time infinitely, the while-loop will keep looping forever. This is known as an infinite loop. It is possible to break out of a while-loop using the keyword `break`. For example:

```
4  int a = 0; // variable a starts at 0
5  while (true) { // while-loop condition is always met
6      // this while-loop would be an infinite loop without a break conditional
7      if (a == 10) { // if a is equal to 10
8          break; // break out of the while-loop
9      }
10     a += 1; // increment a by 1
11 }
12 // This while-loop repeats 10 times. Sounds familiar?
```

while-loop vs for-loop

```
4  int i = 3; // start
5      while (i < 7) { // stop
6          cout << i << " ";
7          i++; // step
8      }
```

```
4      // for (start, stop, step)
5      for (int i = 3; i < 7; i++) {
6          cout << i << " ";
7      }
8      // Output for both: 3 4 5 6
```

Although both loops create the same output, the for-loop reads better and it is preferred in scenarios like this. In both cases, the stop condition is a boolean that needs to be true to keep repeating the loop. If it is false, the loop ends.

Introducing the do-while loop

The code inside a while-loop only executes if the loop condition is met. There are scenarios when you want to execute the code at least once before checking whether or not the condition is met. In such cases, a do-while loop is handy:

```
4 // get the age from user at least once and keeps asking for age if entered age is negative
5 int age = 0; // initialize age variable
6 do {
7     cin >> age; // get age from user input
8 } while (age < 0); // repeat if the entered age is negative
```

The same program can be written using only a while-loop. However, it would require writing more lines of code as you have to get the input once before the while-loop

Introducing Arrays

An `int` variable allows us to store a single integer value. What if we need to store multiple `int` values in a list? We have to use arrays:

```
4  int myArray[3];    // create an empty integer array of size 3
5  myArray[0] = 12;   // set index 0 equal to 12
6  myArray[1] = 21;   // set index 1 equal to 21
7  myArray[2] = 2000; // set index 2 equal to 2000
```

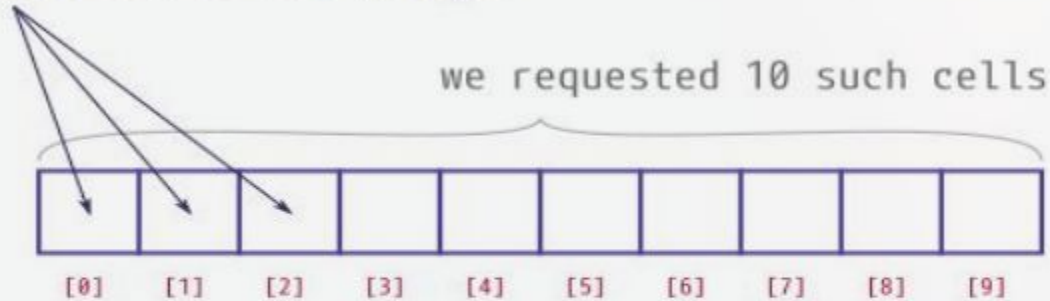
The same result can also be achieved using the following syntax:

```
int myArray[3] = { 12, 21, 2000 }; // create an integer array containing 12,21,200
```

Visualizing Arrays

```
int myData[10];           // empty integer array of size 10
```

each cell stores one integer



the cells are indexed from 0 to 9

Iterating through an Array using a for-loop

Items stored in an array can be accessed by their index. Using a for-loop it is possible to go through the entire array and access all items:

```
4  int myArray[3] = { 12, 21, 2000 }; // create an integer array containing 12,21,2000
5  for (int i = 0; i < 3; i++) { // i will be 0,1,2
6      cout << myArray[i] << " "; // print item in index i of myArray
7  }
8  // output: 12 21 2000
```

Iterating through an array can have other applications than just printing the array.

Enhanced for loops

Lastly, if you want to loop through an array, and you don't need to access the indices, you can use the following notation

```
int myArray[3] = { 12, 21, 2000 }; // create an integer array containing 12,21,2000
for (int i : myArray){ // i will be 12, 21, 2000
    cout << i << " "; // prints the value of i
}
```

Which does the same as

```
4 int myArray[3] = { 12, 21, 2000 }; // create an integer array containing 12,21,2000
5 for (int i = 0; i < 3; i++) { // i will be 0,1,2
6     cout << myArray[i] << " "; // print item in index i of myArray
7 }
8 // output: 12 21 2000
```

The modulo operator (%)

The modulo operator (%) computes the remainder of a division operation. For example, $37 \% 10$ returns 7, because this is the remainder of 37 when divided by 10. This can be very useful for different tasks. Here's an example:

```
1  int num;
2  cin << num;           // get input from user
3  if (num % 2 == 0) {    // if num divided by 2 has remainder of 0
4      cout << "Even number" << endl; // it is an even number
5  }
6  else {
7      cout << "Odd number" << endl;  // otherwise it is an odd number
8  }
```

Linux Commands

Command	Description
<code>pwd</code>	print the current working directory
<code>ls</code>	list files in the current directory
<code>ls path/to/a/directory</code>	list files in the directory
<code>cd path/to/a/directory</code>	change directory
<code>cd ..</code>	go to the parent directory (one level up)
<code>mkdir newdirectoryname</code>	create new directory
<code>cp file1 file2</code>	copy file1 and call the copy file2
<code>mv file1 file2</code>	rename (move) file1 to file2
<code>rmdir directoryname</code>	remove empty directory
<code>rm file</code>	remove file

Compiling code with g++ in the terminal

In Linux/Unix terminals, `g++` allows compilation of C++ code. Let's say that we have some code written in a file called `code.cpp`. If we compile this using `g++`, `g++` will create an executable file.

We can configure the name of this file during compilation:

- Compile with default executable name: `g++ code.cpp`
- Compile with custom executable name: `g++ -o myprog code.cpp`

Running the executable:

- Default executable: `./a.out`
- Custom executable: `./myprog`

Which text editor to use?

There are different IDE and text editors out there that can be used with C++. Most common ones are: VS code, Vim, eclipse, sublime text, gedit etc.

Try out some of them and see which one works for you. I personally recommend using VS code since it works with different programming languages and supports different plugins.

Check the course webpage daily for updates and materials. Each homework and lab prepares you for the quizzes and exams. Keep up with the deadlines and due dates for all assignments and you will be prepared for the exams and quizzes.

Course Webpage: https://tong-yee.github.io/135/2025_spring.html

Good Coding Practices

Computer Science is a very competitive field. What separates a good programmer from the competition is how readable their code is. This includes being consistent in syntax, descriptive variable names, proper documentation and comments etc.

While your code might compile and run, it maybe difficult to understand for other programmers. Being consistent will help both you and the UTA that is trying to help you debug your code. Try to follow this style-guide:

https://maryash.github.io/135/worked_examples/style_guide.html